0

(Task 1 of 14) In this tutorial, we will learn more about **variable assignments** and **mutable variables**.

Lispy |

(Task 2 of 14) What is the result of running this program?

Lispy [Run ▶]

```
(defvar x 1)
(deffun (f n)
  (+ x n))
(set! x 2)
(f 30)
```

Scala 3

```
var x = 1
def f(n : Int) =
  x + n
x = 2
println(f(30))
```

2

`32`

3

You predicted the output correctly 🎉

The program binds `x` to `1` and then defines a function `f`. `x` is then bound to `2`. So, `(f 30)` is `(+ x 30)`, which is `32`.s

Click here to run this program in the Stacker.

Lispy |

(Task 3 of 14) What is the result of running this program?

Lispy [Run ▶]

```
(set! foobar 2)
foobar
```

Pseudo

```
foobar = 2
print(foobar)
```

⚠️Python behaves differently.

5

`error`

6

You predicted the output correctly 🎉

Mutating an undefined variable (in this case, `foobar`) errors rather than defining the variable. So, this program errors.

Click here to run this program in the Stacker.

Lispy |

(Task 4 of 14) What is the result of running this program?

Lispy [Run ▶]

```
(defvar x 12)
```

Pseudo

```
let x = 12
```

```
(deffun (f)
  x)
(deffun (g)
  (set! x 0)
  (f))
(g)
(set! x 1)
(f)
```

```
fun f():
  return x
end
fun g():
  x = 0
  return f()
end
print(g())
x = 1
print(f())
```

`0 1`

8

You predicted the output correctly 🎉

9

There is exactly one variable `x`. The `x` in `(set! x 0)` refers to that variable. Calling `f` evaluates `(set! x 0)`, which mutates `x`. When the value of `x` is eventually printed, we see the new value.

Click here to run this program in the Stacker.

(Task 5 of 14) What did you learn about variable assignment from these programs?

10

not much

11

(Task 6 of 14) - Variable assignments mutate existing bindings and do not create new bindings.

12

- Functions refer to the latest values of variables defined outside their definitions. That is, functions do not remember the values of those variables from when the functions were defined.

Any feedback regarding these statements? Feel free to skip this question.

13

(You skipped the question.)

14

(Task 7 of 14) Please scroll back and select 1-3 programs that make the following point:

15

> Variable assignments mutate existing bindings and do not create new bindings.

You don't need to select *all* such programs.

(*You selected 2 programs*) 16

Okay. How do these programs (4,7) support the point? 17

? 18

(Task 8 of 14) Please scroll back and select 1-3 programs that make the following point: 19

> Functions refer to the latest values of variables defined outside their definitions. That is, functions do not remember the values of those variables from when the functions were defined.

You don't need to select *all* such programs.

(*You selected 1 programs*) 20

Okay. How does this program (1) support the point? 21

. 22

Lispy |

(Task 9 of 14) Although we keep saying "this variable is mutated", the variables themselves are *not* mutated. What is actually mutated is *the binding between the variables and their values*.

Blocks have been a good way to describe these bindings. However, they can't explain variable mutations: a block is a piece of source code, and we can't change the source code by mutating a variable. Besides, blocks can't explain how parameters might be bound differently in different function calls. Consider the following program:

Lispy [Run ▶]

```
(deffun (f n)
  (+ n 1))
(f 2)
(f 3)
```

Python

```
def f(n):
    return n + 1
print(f(2))
print(f(3))
```

In this program, `n` is bound to `2` in this first function call, and `3` in the second. Blocks can't explain how `n` is bound differently because the two calls share the same block: the body of `f`.

What is a better way to describe the binding between variables and their values?

?

24

(Task 10 of 14) **Environments** (rather than blocks) bind variables to values.

25

Similar to vectors, environments are created as programs run.

Environments are created from blocks. They form a tree-like structure, respecting the tree-like structure of their corresponding blocks. So, we have a primordial environment, a top-level environment, and environments created from function bodies.

Every function call creates a new environment. This is very different from the block perspective: every function corresponds to exactly one block, its body.

Any feedback regarding these statements? Feel free to skip this question.

26

?

27

Thank you!

28

(Task 11 of 14) Which language construct(s) create new bindings?

29

- [x] Definitions
- [ ] Variable mutations (e.g., `(set! x 3)`)
- [ ] Variable references
- [ ] Function calls

30

You should also have chosen Function calls

31

Both definitions and function calls create bindings. Variable assignment mutates existing bindings but doesn't create new bindings.

(Task 12 of 14) Which language construct(s) mutate bindings?

32

- [x] Definitions
- [ ] Variable mutations (e.g., `(set! x 3)`)
- [ ] Variable references
- [x] Function calls

33

34

You should also have chosen Variable mutations (e.g., `(set! x 3)`)

You should NOT have chosen Definitions and Function calls

Only variable assignments mutate bindings. Definitions and function calls create new bindings but don't mutate existing bindings.

Lispy | 🎲

(Task 13 of 14) Here is a program that confused many students

Lispy [Run ▶️]

```
(defvar x 5)
(deffun (f x y)
  (set! x y))
(f x 6)
x
```

JavaScript

```
let x = 5;
function f(x, y) {
  x = y;
  return;
}
console.log(f(x, 6));
console.log(x);
```

Please

1. Run this program in the stacker by clicking one of the [Run ▶️] buttons above;
2. The stacker would show how this program produces its result(s);
3. Keep clicking ⏭ Next until you reach a configuration that you find particularly helpful;
4. Click 🔗Share This Configuration to get a link to your configuration;
5. Submit your link below;

36

https://smol-tutor.xyz/stacker/?syntax=Lispy&randomSeed=smol-tutor&hole=%E2%80%A2&nNext=5&program=%0A%28defvar+x+5%29%0A%28deffun+%28f+x+y%29%0A++%28set%21+x+y%29%29%0A%28f+x+6%29%0Ax%0A&readOnlyMode=

37

(Task 14 of 14) Please write a couple of sentences to explain how your configuration explains the result(s) of the program.

38

set! changes the variable in f's environment, not the top-level environment

---

39

Let's review what we have learned in this tutorial.

- Variable assignments mutate existing bindings and do not create new bindings.
- Functions refer to the latest values of variables defined outside their definitions

- Functions refer to the latest values of variables defined outside their definitions. That is, functions do not remember the values of those variables from when the functions were defined.

Environments (rather than blocks) bind variables to values.

Similar to vectors, environments are created as programs run.

Environments are created from blocks. They form a tree-like structure, respecting the tree-like structure of their corresponding blocks. So, we have a primordial environment, a top-level environment, and environments created from function bodies.

Every function call creates a new environment. This is very different from the block perspective: every function corresponds to exactly one block, its body.

You have finished this tutorial 🎉

Please print the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1781413041696